

OMU Mini Project Final Report

Project Title: APU Campus Wide Drinks and Snack Ordering Kiosk System

Module: Object Modelling with UML

Submission: UML report and Java skeletal implementation

Institution: Asia Pacific University of Technology and Innovation

Year: 2026

Group Members: *Add names and student IDs*

This report presents an object-oriented design for the APU drinks and snack ordering kiosk system. It uses the uploaded UXF diagrams as the main design basis, includes rendered versions of the diagrams, and documents a Java skeleton implementing the selected design patterns.

Table of Contents

1. Introduction and Project Plan
2. Analysis and UML Design
3. Design Pattern Refinement
4. Implementation
5. Scenario Walkthrough
6. Compliance Assessment
7. Critical Appraisal
8. Conclusion and Future Enhancements
9. References

1. Introduction and Project Plan

Asia Pacific University plans to deploy a kiosk system that allows students, staff, and lecturers to order drinks and snacks before collecting them from the snack booth. The system is intended to reduce queue pressure around the booth by letting users browse products, customise selected items, pay through the AP card system, and send confirmed orders to the preparation area.

The first deployment contains five kiosks, but the design should remain suitable if the campus later expands the system to ten kiosks. This creates two important design concerns: product information must be maintained centrally, and all kiosks must receive consistent price and availability updates.

1.1 Project Objectives

- Model the kiosk scenario using UML use case, class, and sequence diagrams.
- Identify the main users, system responsibilities, and runtime interactions.
- Refine the class design with two to three suitable object-oriented design patterns.
- Implement the selected patterns in a Java skeletal prototype.
- Evaluate the design for maintainability, extensibility, and coursework suitability.

1.2 Scope and End Users

The primary end users are students, staff, and lecturers who place orders through kiosks. Admin staff update product prices and availability. Preparation staff receive order notifications and prepare the selected drinks and snacks. The AP card service supports identification and payment.

1.3 Major Functions

- Identify a user and show their current AP card balance.
- Display available drinks and snacks.
- Allow customisation of drinks and snacks.
- Create and submit an order.
- Deduct the order amount from the AP card balance.
- Notify the preparation area of paid orders.
- Log kiosk transactions.
- Update product prices and availability centrally.
- Broadcast menu changes to connected kiosks.

2. Analysis and UML Design

2.1 Use Case Model

Source: 01_use_case_diagram.uxf

Rendered image: images/01_use_case_diagram.svg

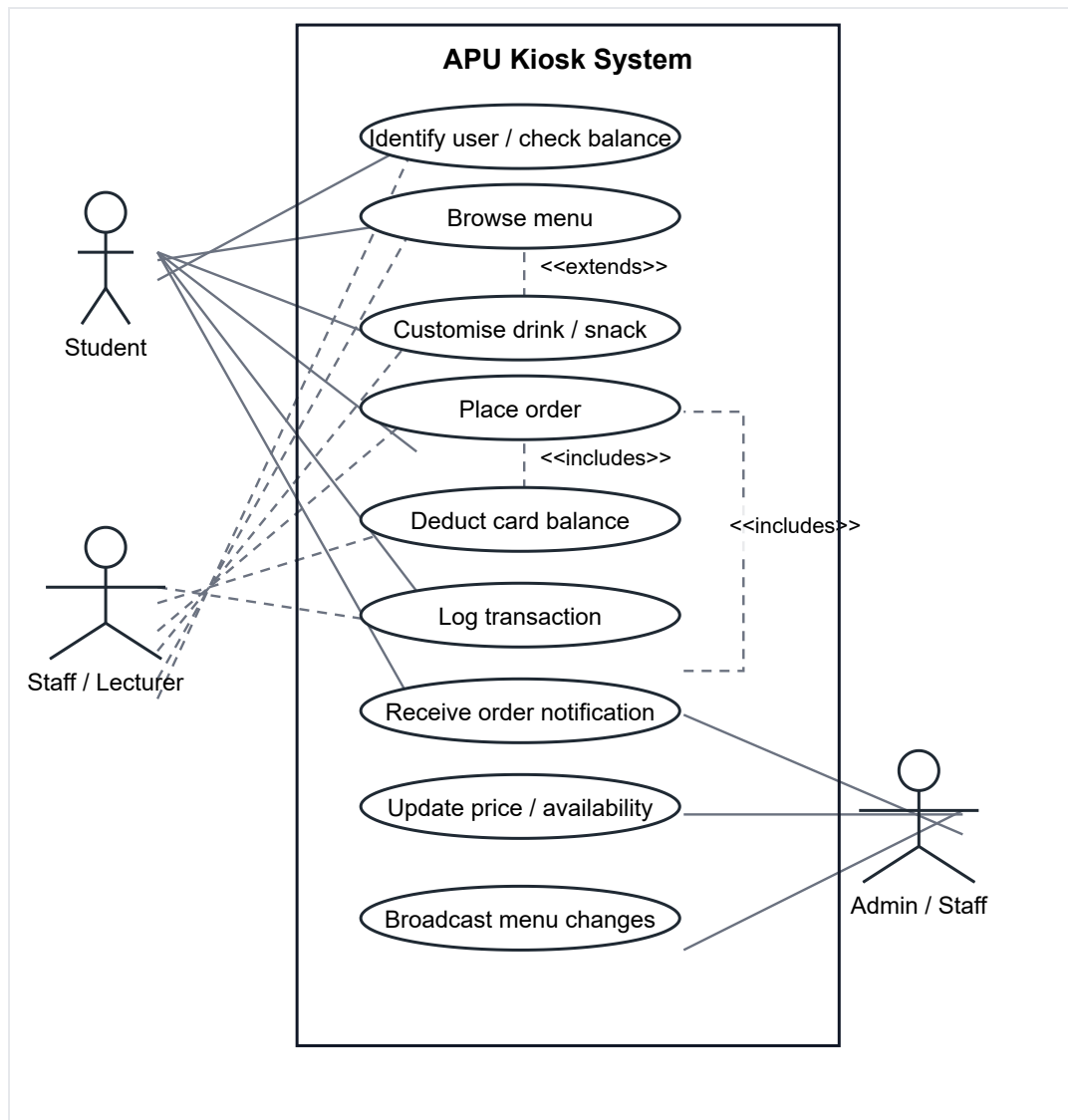


Figure 1 shows the main actors and use cases. Students and staff use the ordering workflow, while admin staff manage product updates and order notifications. The use cases also show supporting operations such as payment deduction, transaction logging, and broadcasting menu changes.

2.2 Main Use Case Descriptions

Use Case	Description	Main Actor
Identify user / check balance	The kiosk reads the AP card, authenticates the card ID, and displays user details with the available balance.	Student, staff, lecturer
Browse menu	The user views the current list of drinks and snacks available at the kiosk.	Student, staff, lecturer
Customise drink / snack	The user chooses options such as drink size, sugar level, milk level, ice level, toppings, sandwich ingredients, or dressing.	Student, staff, lecturer
Place order	The system creates an order from selected items, calculates the total, requests payment, and confirms the order if payment succeeds.	Student, staff, lecturer
Deduct card balance	The payment logic deducts the order total from the AP card balance.	AP card payment service
Log transaction	The kiosk records the order and payment event for traceability.	Kiosk system
Receive order notification	Paid order details are made available to snack booth staff for preparation.	Preparation/admin staff
Update price / availability	Admin staff change product price or availability from the admin system.	Admin staff
Broadcast menu changes	Connected kiosks receive the latest menu state after an admin update.	Admin system

2.3 Initial Class Diagram

Source: 02_class_diagram_no_patterns.uxf

Clean redraw: diagrams/02_class_diagram_no_patterns.mmd

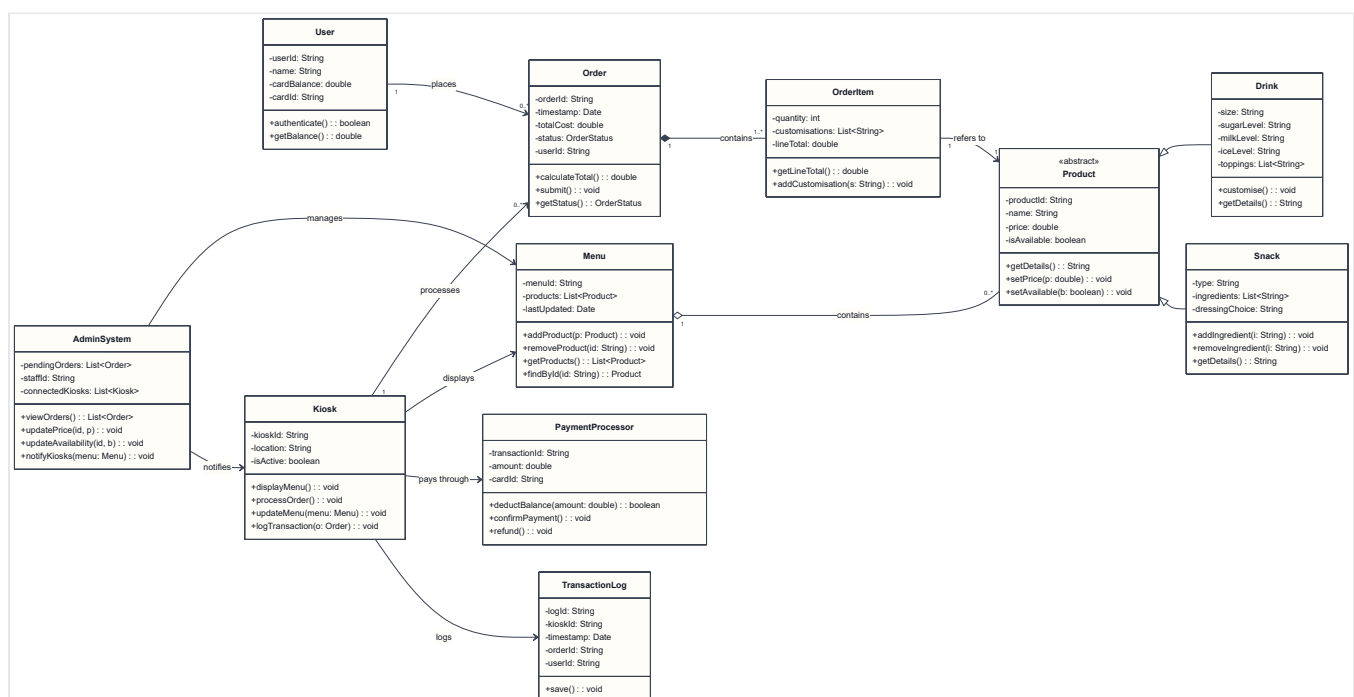


Figure 2 models the system before design patterns are introduced. User, Kiosk, Order, OrderItem, Product, Drink, Snack, Menu, PaymentProcessor, TransactionLog, and AdminSystem form the core object model.

The initial design is understandable, but some responsibilities are still broad. For example, Kiosk processes orders, updates menus, and logs transactions. The refined design improves this by separating payment behaviour, menu update notification, and shared logging/menu management.

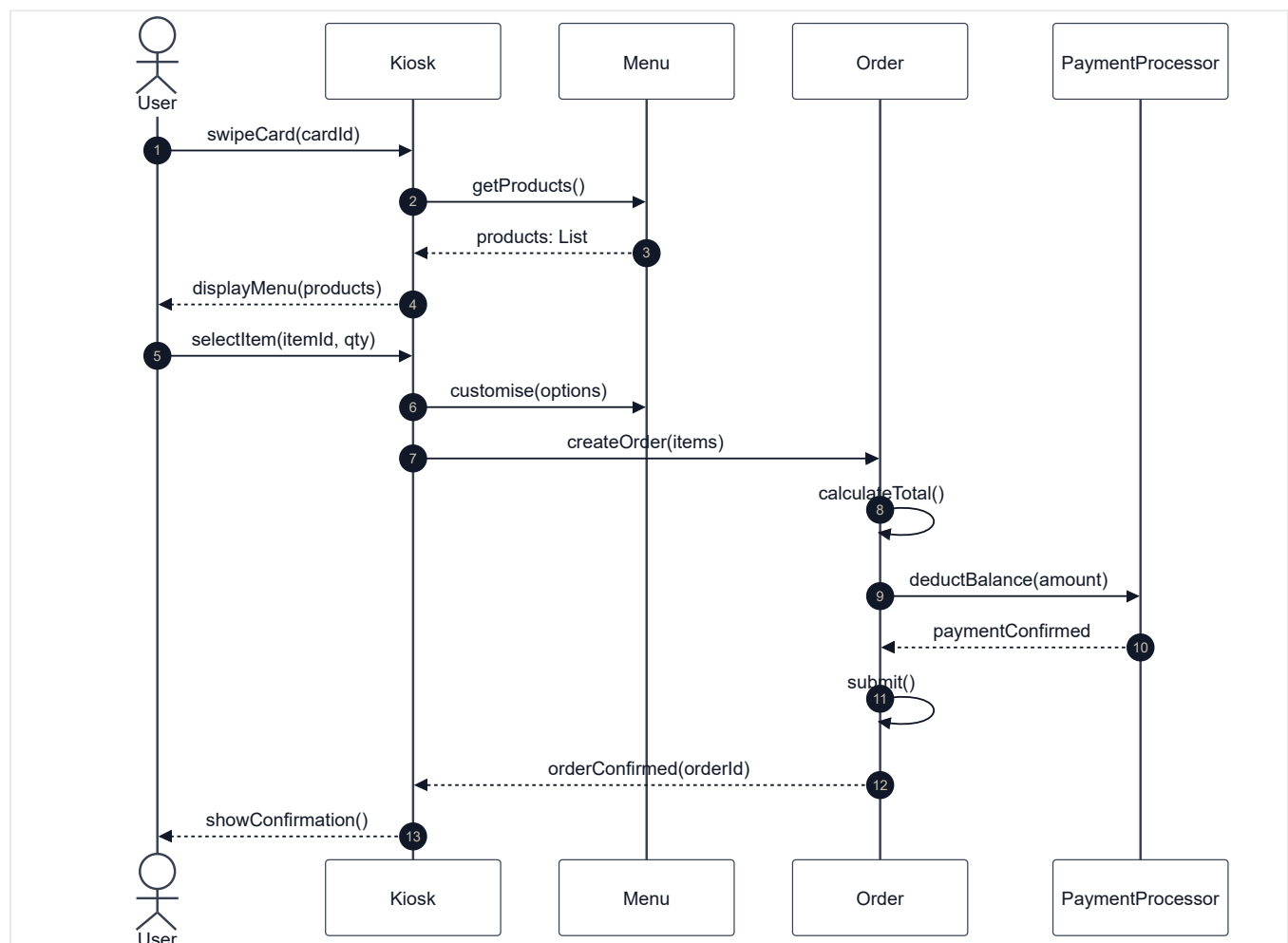
2.4 Sequence Diagrams

The following three sequence diagrams document the main runtime flows required by the assignment.

2.4.1 Place Order

Source: 03_sequence_place_order.uxf

Clean redraw: diagrams/03_sequence_place_order.mmd

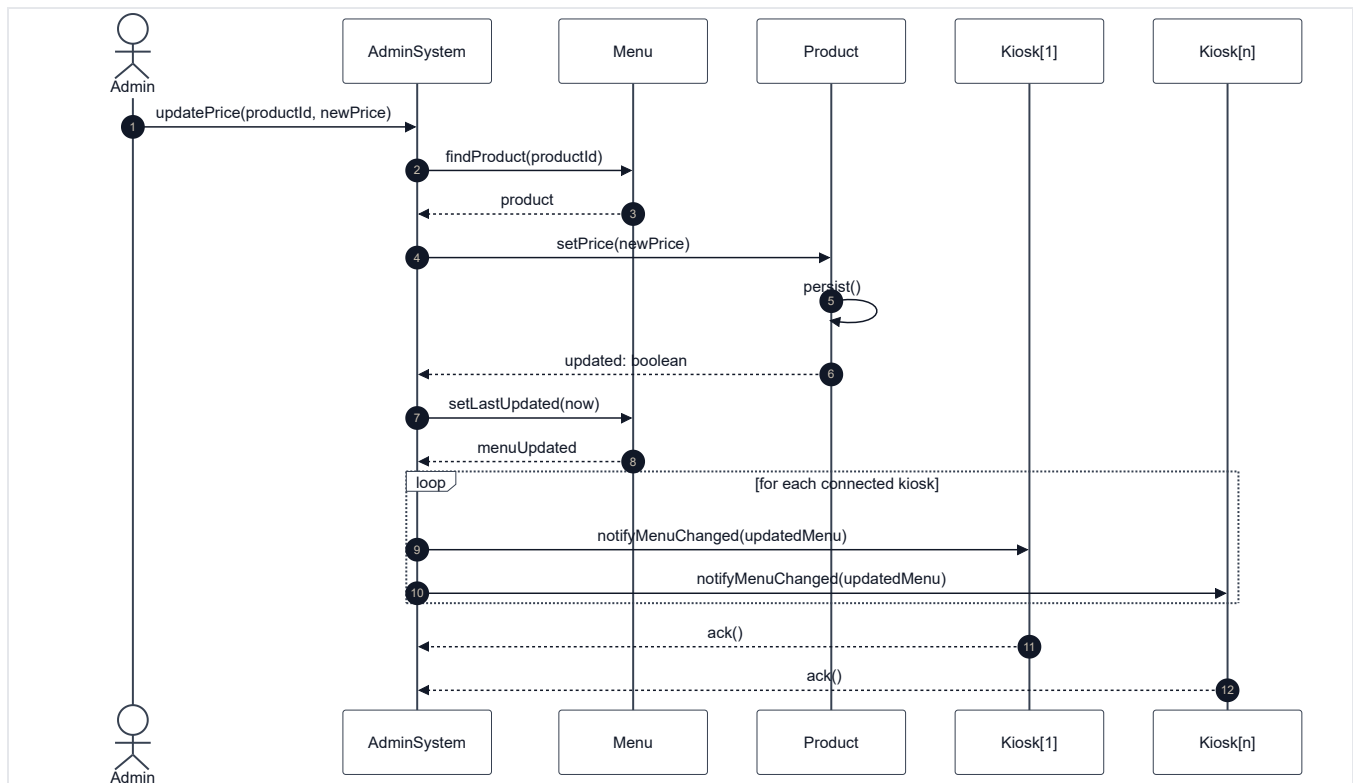


The place order sequence begins when the user swipes an AP card and views the menu. The user selects an item and customisation options, the kiosk creates the order, the order total is calculated, payment is deducted, and the confirmation is displayed.

2.4.2 Update Menu

Source: 04_sequence_update_menu.uxf

Clean redraw: diagrams/04_sequence_update_menu.mmd

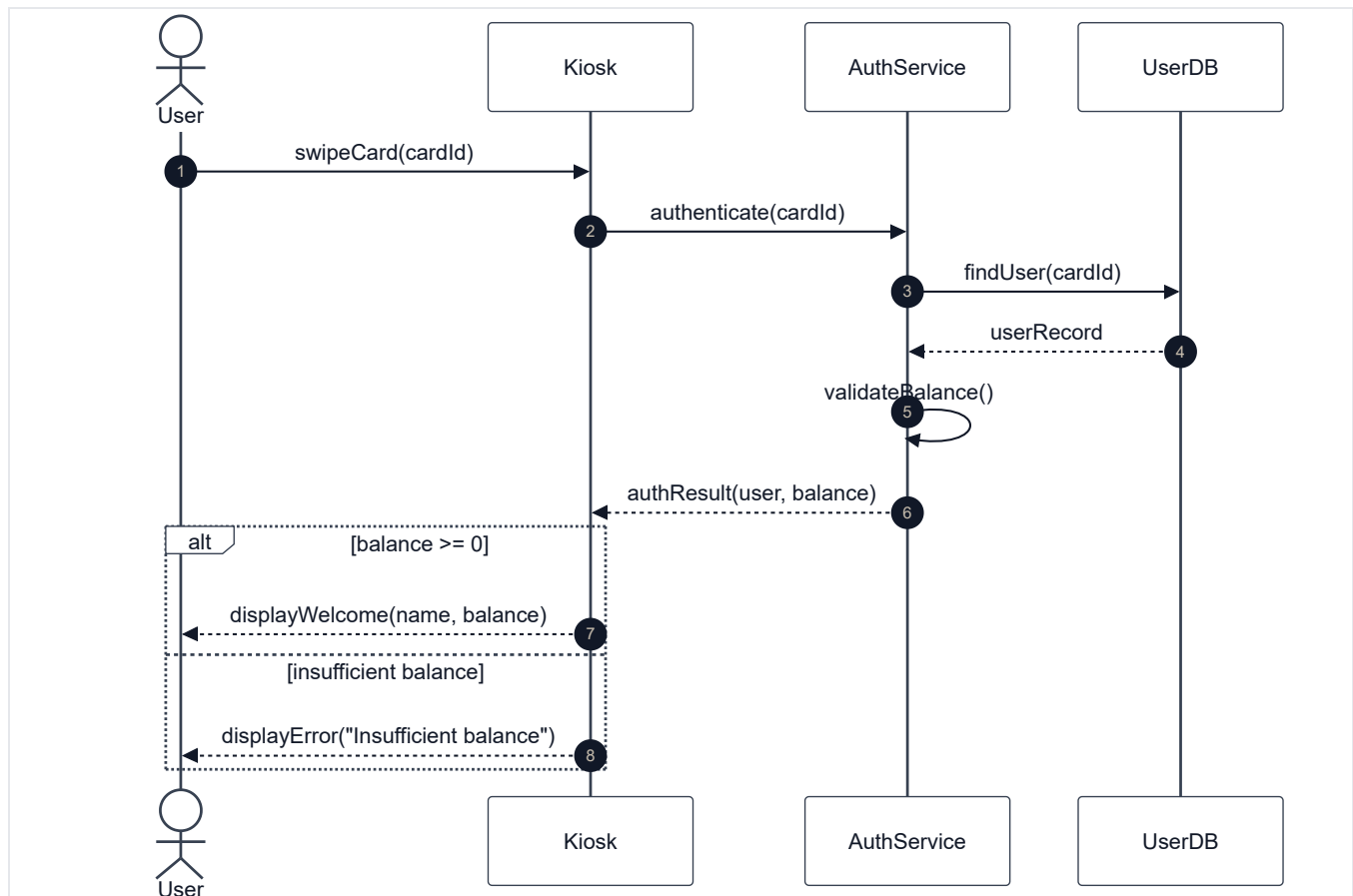


The update menu sequence shows admin staff changing product data centrally. The system finds the product, updates its price or availability, records the update, refreshes the menu timestamp, and notifies connected kiosks.

2.4.3 Authenticate User

Source: 05_sequence_authenticate_user.uxf

Clean redraw: diagrams/05_sequence_authenticate_user.mmd



The authentication sequence shows the kiosk sending a card ID to the authentication service. The service searches for the user record, validates the balance, and returns either a successful authentication result or an error.

3. Design Pattern Refinement

The refined design introduces three patterns: Observer, Strategy, and Singleton. These patterns are chosen because they solve specific problems in the kiosk scenario rather than being added only for demonstration.

3.1 Selected Patterns

Pattern	Role in the Solution	Reason for Selection
Observer	AdminSystem notifies Kiosk observers when menu data changes.	Supports central product updates across multiple kiosks without hard-coding every kiosk into admin logic.
Strategy	Kiosk depends on PaymentStrategy, with APCardPayment as the current implementation and QRCodePayment as a future option.	Keeps payment behaviour replaceable if the university later adds another cashless method.
Singleton	MenuManager and TransactionLogger provide shared access points for menu management and transaction logging.	Helps coordinate shared services that should not have conflicting instances in the skeleton design.

3.2 Refined Class Diagram with Patterns

Source: 06_class_diagram_with_patterns.uxf

Clean redraw: diagrams/06_class_diagram_with_patterns.mmd

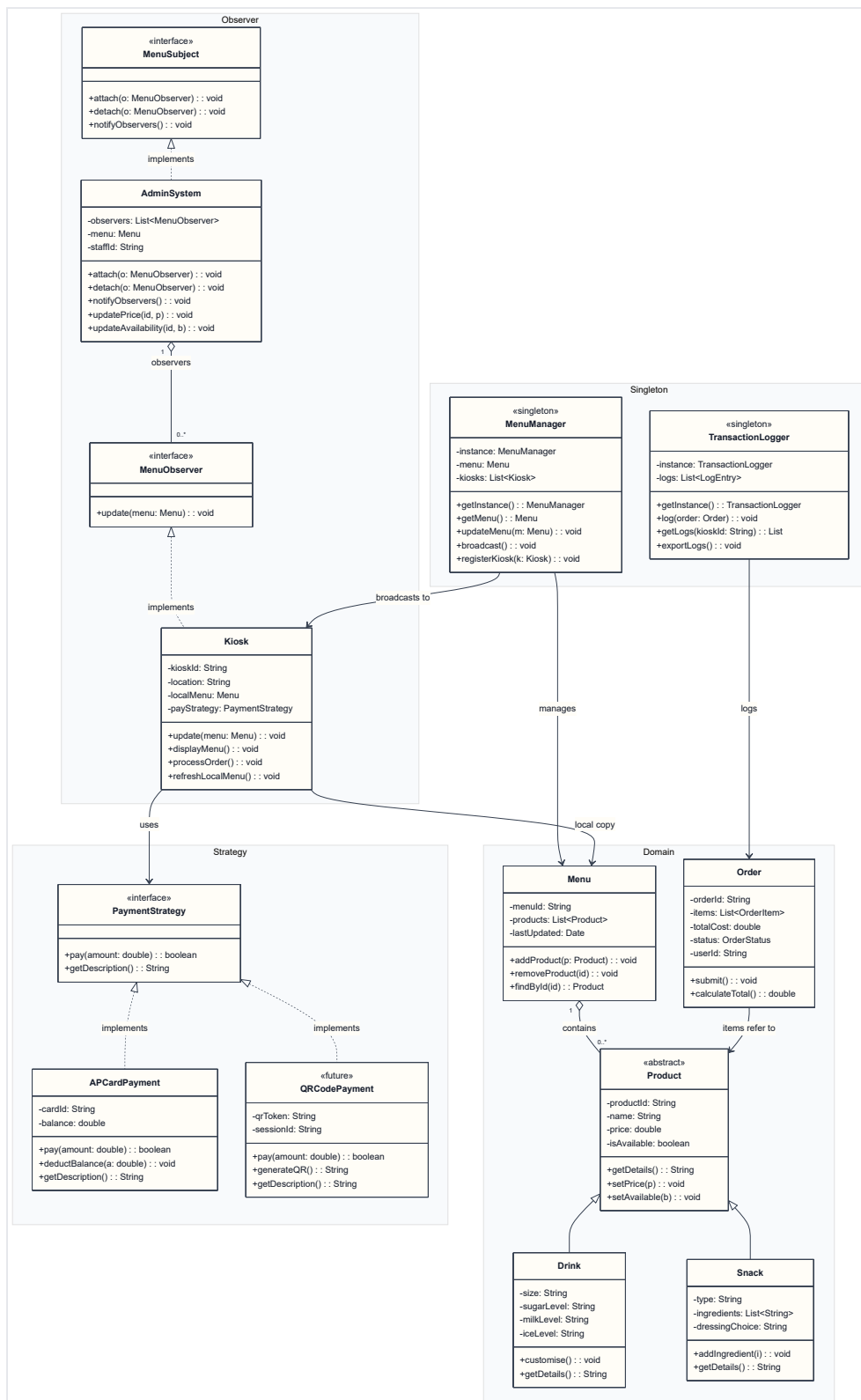


Figure 6 separates the design into pattern-focused areas. The Observer interfaces define menu synchronisation. The Strategy interface defines interchangeable payment behaviour. The Singleton classes centralise shared menu and log access.

3.3 Pattern Interaction Diagrams

The uploaded UXF files include the refined class diagram with patterns. To make the pattern behaviour clearer, two additional sequence diagrams show how the selected patterns interact at runtime.

3.3.1 Observer Menu Broadcast

Source: diagrams/07_sequence_observer_menu_broadcast.mmd

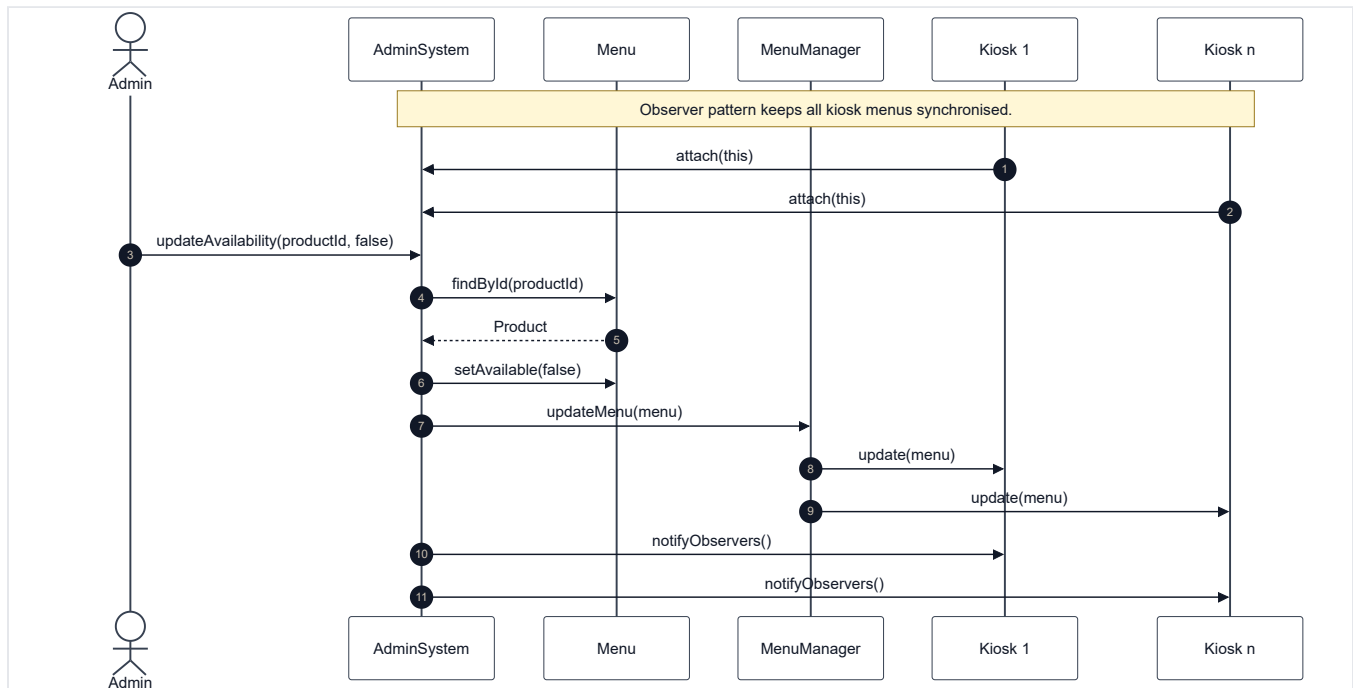


Figure 7 shows kiosks registering for menu updates. When an admin changes product availability, the central menu state is updated and the kiosks receive the change through observer notifications.

3.3.2 Strategy and Singleton Payment Logging

Source: diagrams/08_sequence_strategy_singleton_payment_logging.mmd

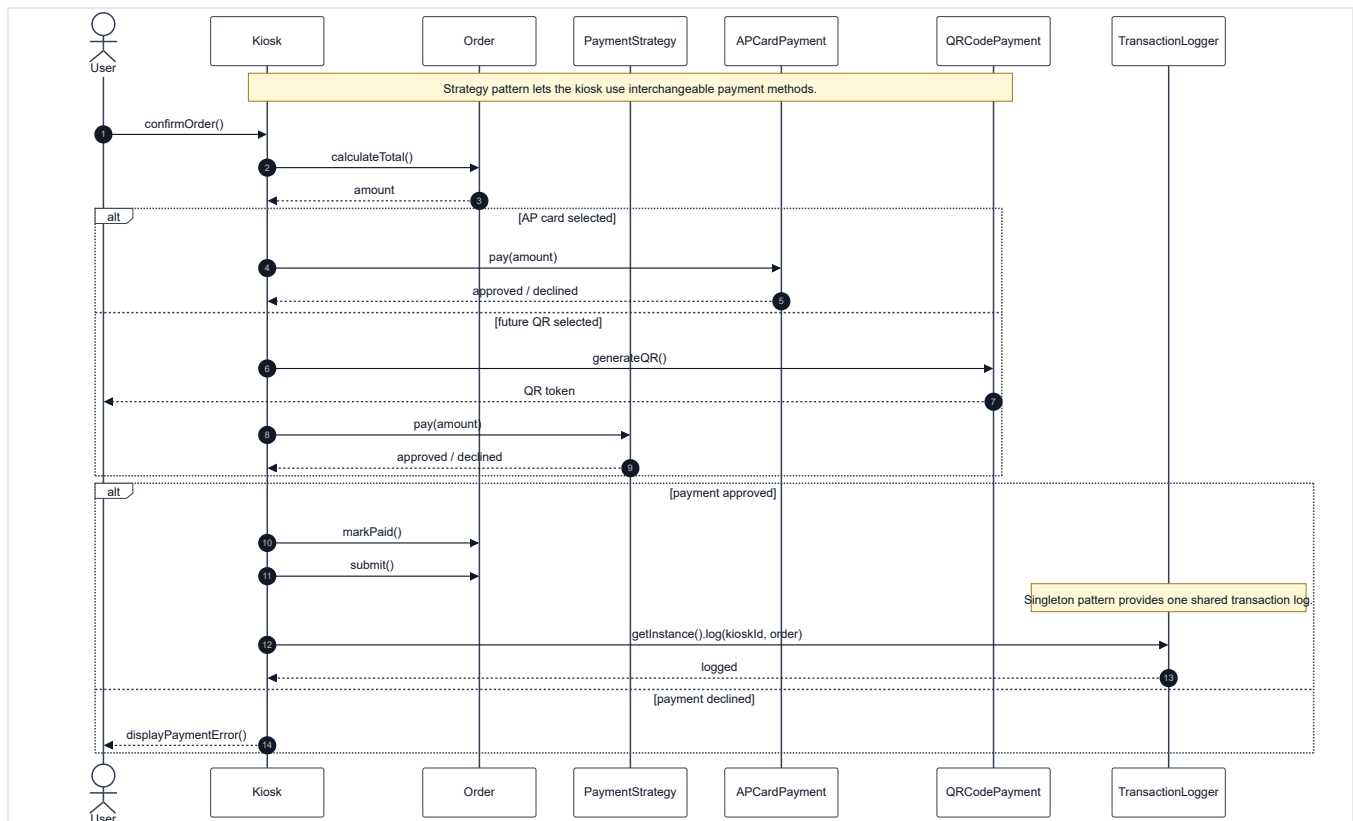


Figure 8 shows the kiosk using a payment strategy to process the order. If payment succeeds, the order is submitted and the shared transaction logger records the event.

4. Implementation

4.1 Technical Structure

The Java implementation is skeletal and focuses on the selected design patterns. It is organised as a Maven project under `src/main/java/edu/apu/kiosk`.

```
src/main/java/edu/apu/kiosk
|-- app
|   |-- DemoApplication.java
|-- domain
|   |-- Drink.java
|   |-- LogEntry.java
|   |-- Menu.java
|   |-- Order.java
|   |-- OrderItem.java
|   |-- OrderStatus.java
|   |-- Product.java
|   |-- Snack.java
|   |-- User.java
|-- observer
|   |-- MenuObserver.java
|   |-- MenuSubject.java
|-- service
|   |-- AdminSystem.java
|   |-- AuthResult.java
|   |-- AuthService.java
|   |-- Kiosk.java
|   |-- UserDatabase.java
|-- singleton
|   |-- MenuManager.java
|   |-- TransactionLogger.java
|-- strategy
|   |-- APCardPayment.java
|   |-- PaymentStrategy.java
|   |-- QRCodePayment.java
```

4.2 Implemented Pattern Evidence

Code Area	Evidence
observer/MenuSubject.java and observer/MenuObserver.java	Define the subject/observer contract used for menu update notifications.
service/AdminSystem.java	Implements the subject role by attaching, detaching, and notifying menu observers.
service/Kiosk.java	Implements the observer role and receives menu updates. It also depends on PaymentStrategy.
strategy/PaymentStrategy.java	Defines the interchangeable payment interface.
strategy/APCardPayment.java and strategy/QRCodePayment.java	Provide current and future payment strategies.
singleton/MenuManager.java	Provides a shared menu manager instance.
singleton/TransactionLogger.java	Provides a shared transaction logging instance.
app/DemoApplication.java	Demonstrates authentication, menu setup, order creation, payment, observer registration, and logging.

4.3 Verification

The diagram rendering scripts were executed successfully and produced both SVG and PNG outputs in images/. The local terminal does not currently expose java, javac, or mvn, so full Java compilation could not be executed in this environment. The source structure and code are prepared for a Java 17 Maven environment.

5. Scenario Walkthrough

5.1 Customer Ordering Flow

1. The user swipes an AP card at the kiosk.
2. The authentication service validates the card and returns the user balance.
3. The kiosk displays available menu products.
4. The user selects drinks or snacks and adds customisation options.
5. The order total is calculated.
6. The selected payment strategy processes the payment.
7. If payment succeeds, the order is submitted.
8. The transaction logger stores the order event.
9. The preparation area receives the confirmed order information.

5.2 Admin Update Flow

1. Admin staff update a product price or availability status.

2. The admin system changes the central menu data.
3. The menu manager refreshes the shared menu state.
4. Registered kiosks receive the update.
5. Each kiosk refreshes its local menu view.

6. Compliance Assessment

Assignment Requirement	Status	Evidence
High-level use case diagram with supporting descriptions	Satisfied	Figure 1 and section 2.2
Class diagram representing the system design	Satisfied	Figure 2
Three sequence diagrams for use cases	Satisfied	Figures 3, 4, and 5
Updated class diagram with two to three design patterns	Satisfied	Figure 6
Sequence diagrams showing pattern interaction	Satisfied	Figures 7 and 8
Implement patterns in Java or C++	Satisfied	Java skeleton under src/main/java/edu/apu/kiosk
Brief evaluation of patterns	Satisfied	Section 3 and section 7
Critical appraisal report	Satisfied	Section 7
References	Satisfied	Section 9

7. Critical Appraisal

7.1 Strengths of the Solution

The solution separates user interaction, payment, menu updates, authentication, and logging into different areas of responsibility. This makes the design easier to understand and maintain than a single kiosk class that performs every task.

The design also fits the assignment scenario. Centralised menu updates support the requirement that price and availability changes should be made from the snack booth system and then applied across kiosks. The observer relationship between admin/menu logic and kiosks supports the planned expansion from five kiosks to ten.

7.2 Suitability of the Design Patterns

Observer is suitable because several kiosks need to react to a central menu update. Strategy is suitable because payment behaviour may evolve beyond AP card payment. Singleton is useful in this

skeletal design for shared menu management and transaction logging, although it should be used carefully in production software.

7.3 Limitations and Risks

The Java implementation is intentionally skeletal. It does not include a real kiosk interface, persistent database, AP card API, network communication, or production deployment model. A real campus deployment would require stronger security, audited admin access, persistent transaction storage, and reliable order notification delivery.

Concurrency is considered through the use of thread-aware collections in some classes, but a production system would still need database transactions or distributed consistency controls to handle simultaneous kiosk orders and admin updates.

7.4 Balanced Evaluation

Overall, the design is appropriate for an UML mini project. It satisfies the functional requirements, demonstrates object-oriented modelling, and applies patterns where they address real design concerns. Its main weakness is prototype depth rather than conceptual structure.

8. Conclusion and Future Enhancements

The proposed kiosk system design addresses the main coursework requirements: identifying users, browsing and customising products, placing orders, processing AP card payment, logging transactions, notifying preparation staff, and updating product data centrally.

The refined design improves maintainability through Observer, Strategy, and Singleton. These patterns support multi-kiosk synchronisation, payment flexibility, and shared service management.

Future improvements could include a graphical kiosk interface, persistent storage, full AP card integration, automated tests, reporting dashboards, staff authentication, and stronger concurrency controls for a real multi-kiosk deployment.

9. References

- Asia Pacific University. (2026). *OMU Mini Project: APU Campus Wide Drinks and Snack Ordering Kiosk System*.
- Gamma, E., Helm, R., Johnson, R., & Vlissides, J. (1994). *Design Patterns: Elements of Reusable Object-Oriented Software*. Addison-Wesley.
- Oracle. (n.d.). *Java Platform Documentation*.
- Mermaid. (n.d.). *Mermaid Diagramming and Charting Tool*.